Instituto Profesional Virginio Gómez
Universidad de Concepción

5 YEARS AVANGADA

INGENIERÍA DE EJECUCIÓN EN COMPUTACIÓN E INFORMÁTICA • SEMANA 9

Arquitectura de Sistemas Avanzada

Diseñando el **Sistema de Gestión de Personal** IPVG con **Herencia** y **Polimorfismo** en **C#** (.NET)

UT 5 & 6 Prep. Certamen 2 | Código: 0568

El Desafío Arquitectónico: Código Redundante

Clase Docente

```
string Rut;
string Nombre;
decimal Sueldo;

void CalificarAlumno();
```

Clase Administrativo

```
string Rut;
string Nombre;
decimal Sueldo;

void GestionarMatricula();
```



¿Qué sucede si IPVG decide agregar "Fecha de Contratación" a todo su personal? ¿Modificamos 20 clases distintas? (Mantenibilidad = 0).

La Herencia como Cimiento Estructural

Al igual que el edificio de la Sede CCP no se construye copiando bloques al azar, un sistema robusto en .NET abstrae las características comunes en una base sólida.



Pisos Superiores
Clases Derivadas: Docente, Administrativo

Cimientos
Clase Base: Empleado

```
public class Empleado { ... }

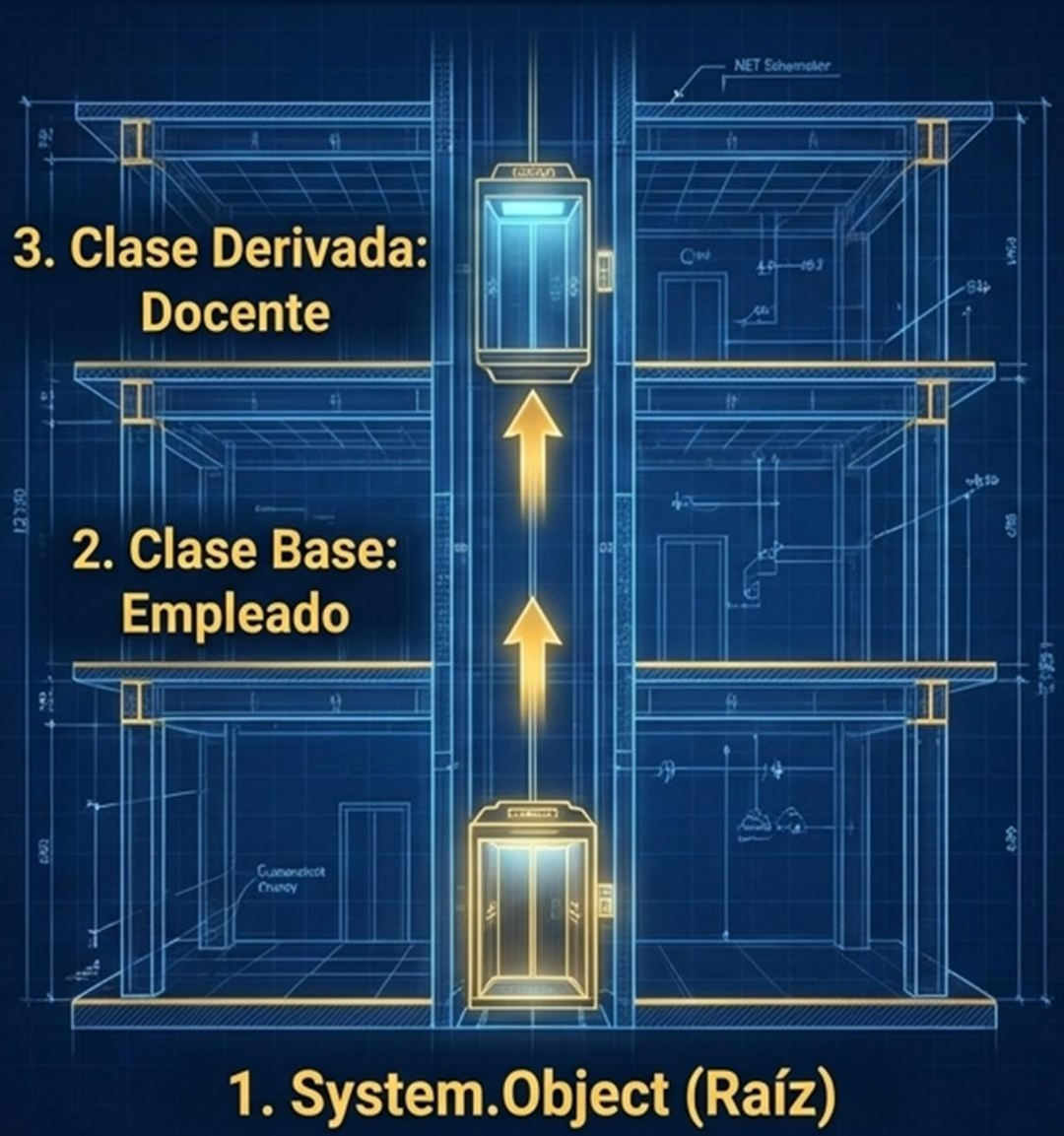
public class Docente : Empleado { ... }
```

La Matriz de Acceso Institucional

	Dentro de la Clase	En Clase Heredada (Hija)	Fuera del Sistema (Program.cs)
`private` [Metáfora: Casillero Personal]	Sí ✔	No ✘	No ✘
`protected` [Metáfora: Sala de Profesores]	Sí ✔	Sí ✔	No ✘
`public` [Metáfora: Hall Central]	Sí ✔	Sí ✔	Sí ✔

La palabra clave `protected` expone los datos *solo* a las ramas de la jerarquía, manteniendo la encapsulación intacta hacia el exterior.

Flujo de Ejecución en Cascada (El Ascensor)



1. **System.Object:** La clase raíz absoluta en C# se inicializa primero.
2. **Clase Base:** Se ejecuta el código del constructor de Empleado.
3. **Clase Derivada:** Finalmente, se ejecuta el código del constructor de Docente.

Insight: La herencia en .NET es una construcción “de adentro hacia afuera” o “de cimientos a techo”.

Comportamientos Distintos en el Mismo Campus

Administrativo



Marca horario fijo de oficina.

Ambos necesitan ejecutar:

```
public void MarcarAsistencia() {
    ...
}
```

Docente



Marca asistencia por bloque de clases.

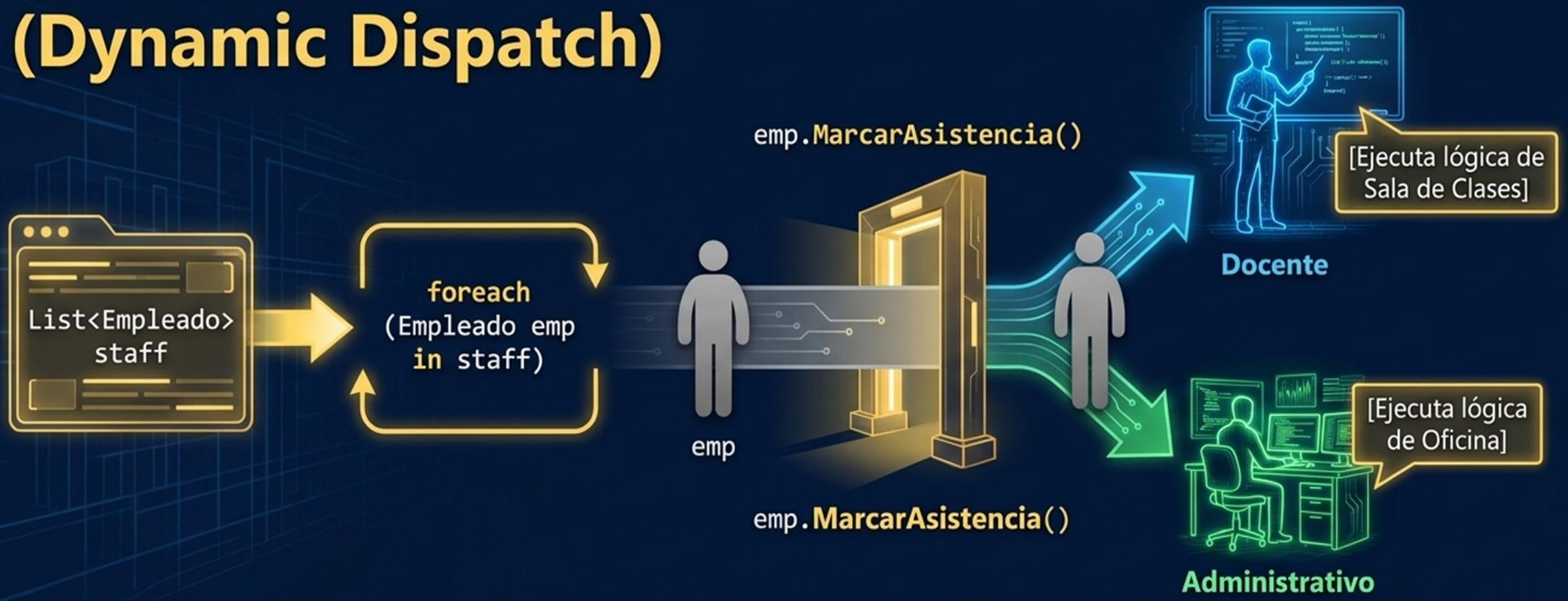
The Business Rule: Todo Empleado debe **registrar asistencia**.

The **Question**: ¿Cómo mantenemos una **única lista** de `List<Empleado>` si cada uno obedece reglas internas completamente distintas al ejecutar la misma acción?

Diferenciando Comportamientos: Sobrecarga vs. Sobrescritura

Sobrecarga (Overload - UT 7)	**Polimorfismo / Sobrescritura** (Override - UT 6)
	
- Resuelto en **Compile-Time**. - El mismo método en la misma clase, pero recibe distintos datos. - Ej. `Imprimir(string)` vs `Imprimir(int)`.	- Resuelto en **Run-Time**. - Jerarquía de herencia. La misma firma exacta del método, pero distintas clases. - La clase hija “aplasta” y reemplaza la implementación de la clase padre.

El Metamorfo Polimórfico (Dynamic Dispatch)



En tiempo de ejecución (Run-time), la plataforma .NET examina el objeto real en memoria. Si el objeto fue instanciado como Docente, ignorará la lógica genérica y ejecutará su propia versión de la realidad.

Poder Arquitectónico: Permite procesar a cientos de empleados distintos con un solo bloque de código genérico.

Las Llaves Maestras: virtual y override

En la Clase Base (Empleado)



```
public virtual void MarcarAsistencia()
{
    ...
}
```

El Permiso: Le dice al compilador "Permito que mis clases hijas cambien esta lógica si lo necesitan".

En la Clase Derivada (Docente)



```
public override void MarcarAsistencia()
{
    ...
}
```

La Acción: Le dice al compilador "Ignora el código de mi padre para esta acción, yo tomaré el control y usaré mi propia lógica".

Extendiendo en lugar de Reemplazar: La palabra base

A veces, no queremos destruir la lógica original del padre, queremos sumar a ella.



Lógica Hija:
Registrar bloque de asignatura

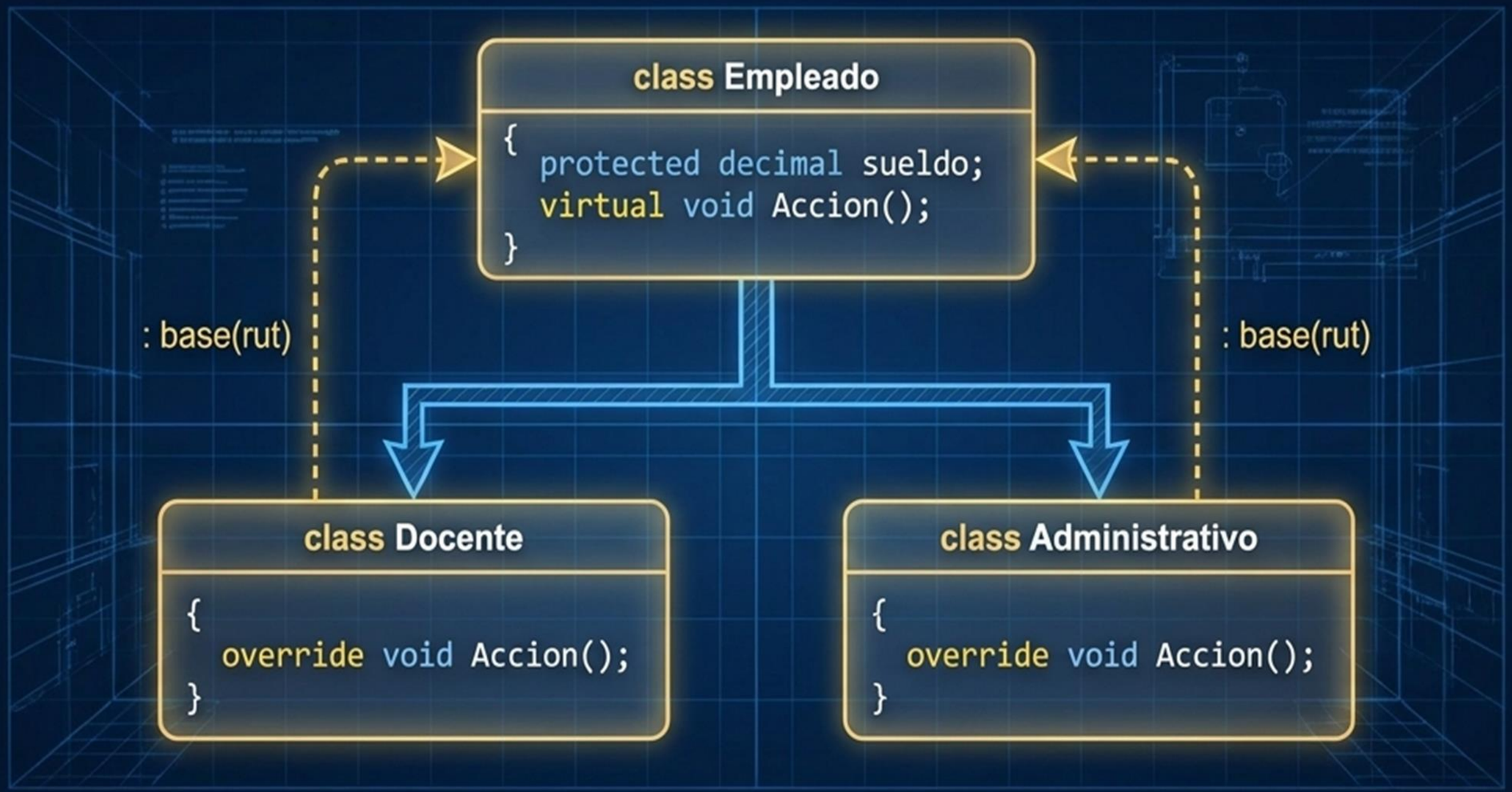
Lógica Padre:
Validar estado activo en sistema

```
public override void MarcarAsistencia()
{
    base.MarcarAsistencia(); // Ejecuta la lógica central del padre primero

    Console.WriteLine('Registrando sala y bloque para el Docente...');
}
```

Nota Arquitectónica: Distinguir claramente entre **': base()'** (usado en firmas de Constructores para instanciación) y **'base.Metodo()'** (usado dentro del cuerpo de la función para extender la lógica de comportamiento).

El Plano Maestro Unificado: Sistema IPVG



Modularidad: Atributos compartidos en la base (protected).

Integridad: Inicialización segura a través de la tubería ancestral (: base()).

Flexibilidad: Comportamientos adaptables mediante Polimorfismo (virtual + override).